# Mobile Application Testing Lab

**Lab Setup**
Attacker Machine: Kali Linux
Target: Android emulator or rooted Android device
Tools – MobSF, Frida, Drozer

## Static Analysis using MobSF

- Start MobSF

  git clone https://github.com/MobSF/Mobile-Security-Framework-MobSF.git
  cd Mobile-Security-Framework-MobSF
  ./setup.sh
  ./run.sh

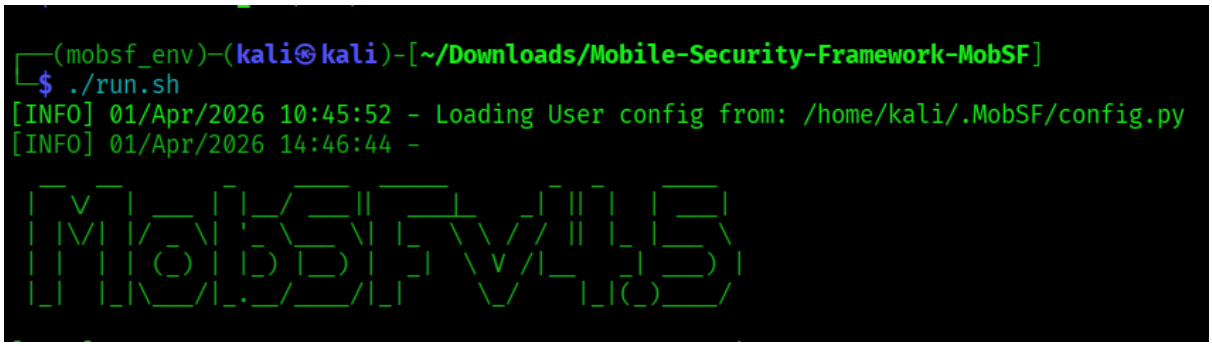


*Figure 1: Run MobSF using command './run.sh'*

Open browser:
http://127.0.0.1:8000

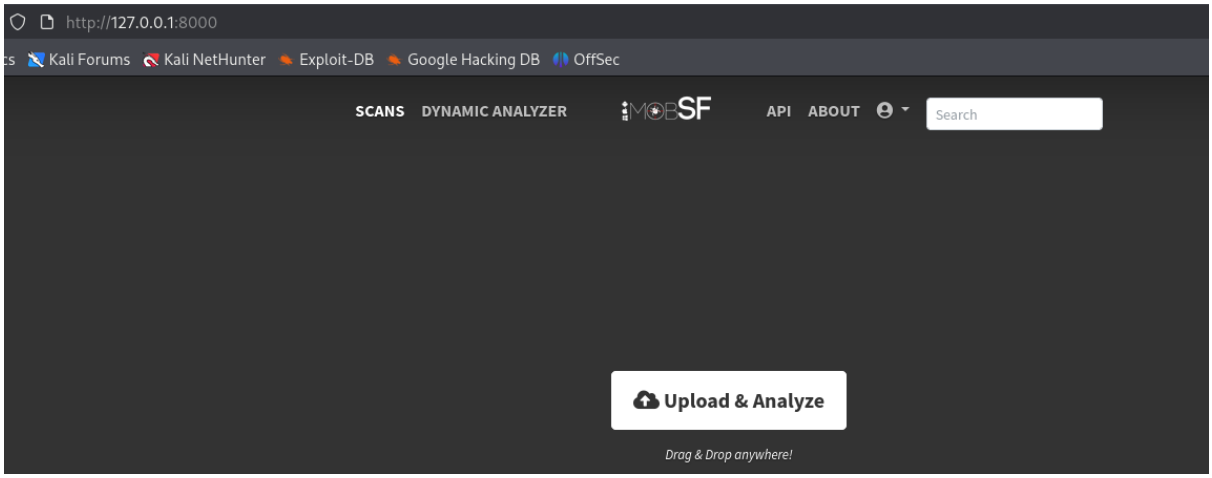


*Figure 2: Open MobSF in Browser*

- Upload the APK file (example: Linkedin.apk).

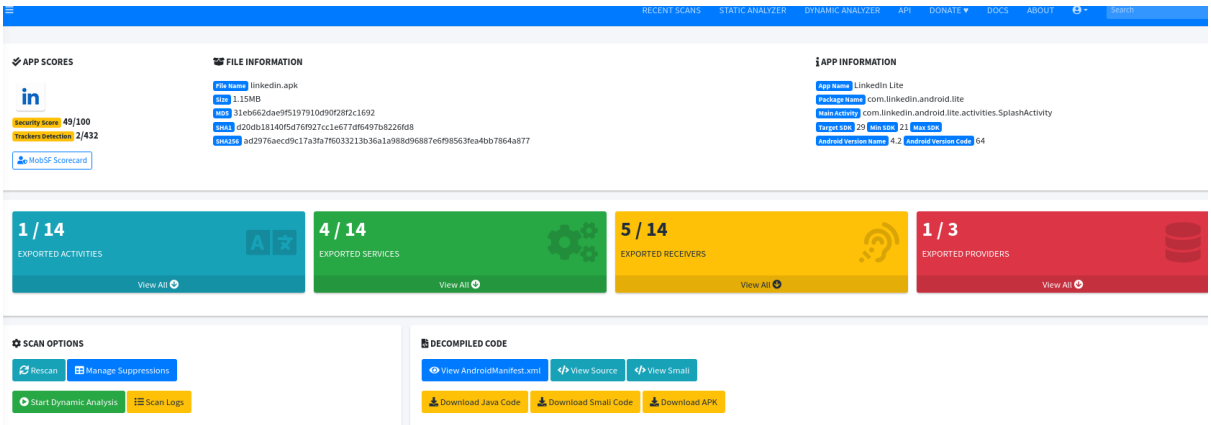


*Figure 3: MobSF apk scan result*

- MobSF will analyse the application and show
  - ➢ tatic analysis using MobSF identified critical vulnerabilities such as insecure storage, hardcoded credentials, and debug configuration enabled.
  - ➢ Dynamic testing with Frida demonstrated that authentication mechanisms can be bypassed by hooking application functions at runtime.
  - ➢ IPC testing using Drozer revealed exported components accessible without proper permissions, leading to potential unauthorized access.

## Dynamic Testing using Frida

- Start Frida server on device:

  adb push frida-server /data/local/tmp/
  adb shell chmod 755 /data/local/tmp/frida-server
  adb shell ./data/local/tmp/frida-server &

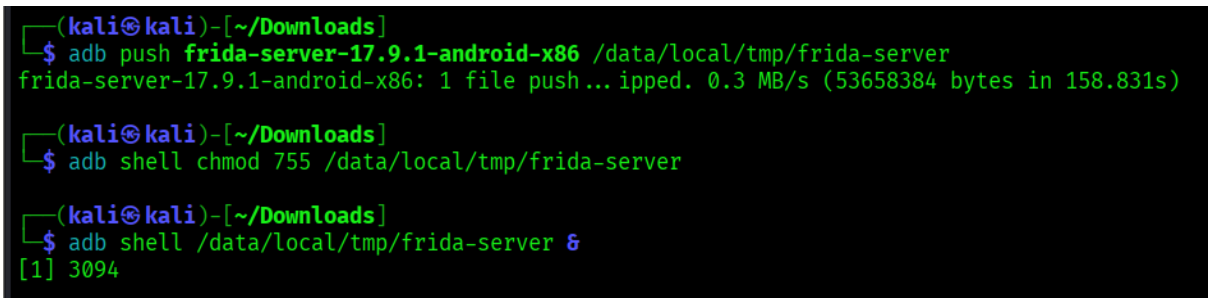


*Figure 4: Start Frida server on device*

- List apps:
  frida-ps -U

```
┌──(kali㉿kali)-[~/Downloads]
└─$ frida-ps -U
 PID  Name
────  ──────────────────────────────────────────────────────────────────
5499  Email
5893  LinkedIn Lite
5212  Messaging
4906  Phone
4537  Settings
5670  Superuser
5833  abb
 557  adbd
4627  android.ext.services
4090  android.hardware.audio@2.0-service
 468  android.hardware.bluetooth@1.0-service.sim
 469  android.hardware.camera.provider@2.4-service
 470  android.hardware.cas@1.1-service
 471  android.hardware.configstore@1.1-service
 472  android.hardware.drm@1.0-service
 473  android.hardware.drm@1.2-service.clearkey
 474  android.hardware.gnss@1.0-service
2669  android.hardware.graphics.allocator@2.0-service
4091  android.hardware.graphics.composer@2.3-service
 477  android.hardware.health@2.0-service.genymotion
 242  android.hardware.keymaster@3.0-service
 478  android.hardware.light@2.0-service
```

*Figure 5: List apps*

- Hook authentication function

  nano test.js

```
Java.perform(function () {
   var Activity = Java.use("android.app.Activity");

   Activity.onResume.implementation = function () {
      console.log("[+] Opened Activity: " + this.getClass().getName());
      return this.onResume();
   };
});
```

```
kali@kali: ~/Downloads
  GNU nano 8.6                                                    test.js
Java.perform(function () {
    var Activity = Java.use("android.app.Activity");

    Activity.onResume.implementation = function () {
        console.log("[+] Opened Activity: " + this.getClass().getName());
        return this.onResume();
    };
});
```

Figure 6: Hook authentication function

- Run test.js
  frida -U -p 5893 -l test.js

```
┌──(kali㉿kali)-[~/Downloads]
└─$ frida -U -p 5893 -l test.js
     ____
    / _  |   Frida 17.9.1 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Phone (id=192.168.56.101:5555)

[Phone::PID::5893 ]->
```

*Figure 7: Run test.js*

- Result
  - Authentication bypassed successfully
  - Application grants access without valid credentials

## IPC Testing (Drozer)

- Start Drozer agent on device

*Figure 8: Start Drozer agent on device*

- Connect from Kali

  drozer console connect

```
┌──(drozer2)─(kali㉿kali)-[~/Downloads]
└─$ drozer console connect
Selecting 46d7e287456a238c (Genymobile Phone 10)

            ..                    ..:.
          ..o..                   .r..
           ..a..  . ....... .   ..nd
             ro..idsnemesisand..pr
             .otectorandroidsneme.
          .,sisandprotectorandroids+.
        ..nemesisandprotectorandroidsn:.
       .emesisandprotectorandroidsnemes..
      ..isandp,..,rotecyayandro,..,idsnem.
      .isisandp..rotectorandroid..snemisis.
      ,andprotectorandroidsnemisisandprotec.
     .torandroidsnemesisandprotectorandroid.
     .snemisisandprotectorandroidsnemesisan:
     .dprotectorandroidsnemesisandprotector.

drozer Console (v3.1.0)
dz> run app.package.list
Attempting to run shell module
com.genymotion.settings (com.genymotion.settings.SettingsServiceApp)
com.android.cts.priv.ctsshim (com.android.cts.priv.ctsshim)
```

*Figure 9: Connect drozer from Kali*

- Enumerate attack surface:

  run app.package.list -f linkedin

```
dz> run app.package.list -f linkedin
Attempting to run shell module
com.linkedin.android.lite (LinkedIn Lite)
```

*Figure 10: Enumerate attack surface*

- Test exported components:

  run app.package.attacksurface com.linkedin.android.lite

```
dz> run app.package.attacksurface com.linkedin.android.lite
Attempting to run shell module
Attack Surface:
  2 activities exported
  5 broadcast receivers exported
  1 content providers exported
  4 services exported
```

*Figure 11: Test exported components*

- Result
  - Exported components accessible without permission
  - Possible privilege escalation or data leakage

## Log Table

| Test ID | Vulnerability | Tool ued | Severity | Target App |
|---|---|---|---|---|
| 001 | Insecure Storage | MobSF | High | Linkedin.apk |
| 002 | Hardcoded Credentials | MobSF | High | Linkedin.apk |
| 003 | Debuggable Enabled | MobSF | Medium | Linkedin.apk |
| 004 | Authentication Bypass | Frida | High | Linkedin.apk |
| 005 | Activity Monitoring | Frida | Medium | Linkedin.apk |
| 006 | Exported Components | Drozer | High | com.linkedin.android.lite |
| 007 | IPC Vulnerability | Drozer | High | com.linkedin.android.lite |
| 008 | Attack Surface Enumeration | Drozer | Medium | com.linkedin.android.lite |

*Table 1: Log table*

## Dynamic Testing – Authentication Bypass using(summary)

Using Frida, the application's authentication function was hooked at runtime. The login verification method was overridden to always return a successful result, bypassing credential checks. This demonstrated weak client-side validation, allowing unauthorized access without valid credentials and highlighting the need for secure server-side authentication mechanisms.

## Lab Summary

This project focused on analyzing and identifying security vulnerabilities in an Android application using **MobSF**, **Frida**, and **Drozer**. Static analysis with MobSF revealed critical issues such as insecure storage of sensitive data, hardcoded credentials, and debug configurations enabled. These vulnerabilities indicate weak security practices and increase the risk of data exposure and reverse engineering attacks.

Dynamic testing using Frida demonstrated that authentication mechanisms could be bypassed by hooking application functions at runtime, allowing unauthorized access. Additionally, IPC testing with Drozer identified exported components accessible without proper permissions, leading to potential data leakage and privilege escalation. Overall, the project highlights the importance of secure coding, proper data encryption, and strict access control to enhance mobile application security.